
Advanced AI Interview System

TECHNICAL ARCHITECTURE & SUMMARY REPORT

Author: Simarpreet Singh

Date: June 21, 2026

System Version: 1.2.0-prod

Environment: Hybrid (Render Backend / Vercel Web UI / SQLite-Postgres Dual Sync)

Status: Deployment Validated & Operational

1. Executive Summary

The **Advanced AI Interview System** is a state-of-the-art, end-to-end recruitment solution designed to automate mock interviews, streamline technical skill vetting, and run rigorous cognitive and behavioral evaluations. The system incorporates multi-modal analytics—tracking client-side facial focus, extracting candidate sentiments in real-time, transcribing high-accuracy speech via OpenAI Whisper, and rendering scoring and interactive feedback powered by Gemini 1.5 Flash (free tier) and GPT-4o.

For recruiters and managers, a detailed administrative panel provides real-time dashboard visualization (leveraging 3D column and donut charting from Highcharts), deep-dive dossier analysis, integrity monitoring logs (capturing tab switching or webcams leaving focus), and access controls. The system is designed with full resilience, defaulting to a local offline SQLite and TF-IDF scoring fallback to ensure zero downtime or cost penalties, with simple tooling to migrate data to a production PostgreSQL database when scaled.

1.1 Core System Capabilities

- **■ Resume Vetting & Parser:** Automated PDF resume parsing with heuristics to capture names, DOB, emails, contact numbers, and qualifications. Includes regex-based branch matching that detects specific engineering fields (AIML, Data Science, CSE, ECE, EEE, etc.).
- **■ Neural Question Selector:** A domain-driven question generator that groups questions by programming language or field and selects behavioral vs technical questions dynamically based on experience levels (fresher vs experienced).
- **■ Multi-Modal Interview Simulator:** An immersive screen layout displaying the live webcam, real-time emotion metrics, and questions. Handles chunked speech capturing, VAD (Voice Activity Detection), and audio-video recording.
- **■■ Integrity & Focus Tracker:** Monitors candidate focus in real-time. Detects cheating flags (e.g. looking away, face missing, tab switching) and appends timestamped incidents to the candidate database records.
- **■ Recruiter Dossier Panel:** Provides detailed statistics, rank hierarchies, direct outreach quick-actions (Gmail & WhatsApp links), resume downloads, video replay channels, and individual evaluation feedback cards.
- **■ DB Sync & Celery Worker Stack:** Configured to run in Docker Compose with Celery workers and Redis broker for heavy Whisper conversions, with a dual DB synchronization script to migrate sqlite local entries to PostgreSQL production tables.

2. Technical Stack & Architecture

The application is engineered using a robust decoupled structure that bridges client-side analytics with containerized backend processing servers.

Layer / Component	Technology / Library	Function / Use Case
Web Server / API	FastAPI (Python 3.10) / Uvicorn	High-performance asynchronous backend API & template router.
AI Transcription	OpenAI Whisper (Local Base / Groq API)	High-accuracy audio speech-to-text transcript generation.
Face & Emotion AI	Face-API.js (Client-side TensorFlow)	Real-time webcam emotion matrix mapping & eye focus checks.
Response Grading	Gemini 1.5 Flash / GPT-4o / Ollama Llama 3	Large Language Model evaluation. Clean JSON grading & feedback.
Offline Grading	Scikit-Learn TF-IDF Cosine Similarity	Local fallback that rates text alignment and length with zero api cost.
Databases	PostgreSQL 15 / SQLite 3 (SQLAlchemy ORM)	Relational schemas for user authentication and session dossiers.
Background Tasks	Celery / Redis Broker & Backend	Handles heavy media processing, transcriptions, and storage uploads.
Cloud Storage	Cloudinary SDK	Persistent host for uploaded resumes and candidate recordings.
Admin Visuals	Highcharts JS (3D Charts & Donut)	Dashboard graphics representing skill counts and scores.

2.1 Code Module Decomposition

- **main.py:** The core orchestrator. Bootstraps FastAPI, configures CORS, mounts static resources, defines endpoints for candidate signup/login, handles upload tasks (PDF parsing, WebRTC audio/video saving), registers background tasks, and provides the administrator controls.
- **models.py:** Defines the database schema via SQLAlchemy. Maps the **User** model (containing account details, parsed resume info, email, phone, age, integrity violations, and full interview recording url) and the child **InterviewSession** model (containing individual question scores, answers, primary emotions, video url, and evaluation feedback).
- **modules/resume_parser.py:** Integrates `pdfplumber` to extract raw text and executes regex boundary models. Matches canonical programming languages (Python, Java, Swift, etc.), DevOps structures, and databases. Contains an extensive qualification matching hierarchy to parse specific degrees (e.g. B.Tech CSE, BCA, B.Sc Data Science, MBBS) and extracts emails, phone numbers, and estimated age.

- **modules/question_generator.py:** Stores a deep question pool categorized by language and difficulty. Classifies candidates into target domains (AI/ML, CS, Electrical, Mechanical, Electronics) based on resume skills and dynamically draws questions, verifying in DB history that a user is never asked duplicate questions.
- **modules/answer_analyzer.py:** Handles response scoring. Checks if custom API keys are present in ``.env`` to execute Gemini 1.5 Flash (via JSON generation schema API calls) or OpenAI GPT-4o, or loops into local Llama 3 via Ollama. If no endpoints are reachable, executes the local TF-IDF Cosine Similarity scoring model, awarding custom bonuses for word counts and applying emotion weightings (e.g. +10% for confidence, -5% for fear).

3. Data Architecture & API Design

The system maintains data integrity through relational tables and handles REST and WebSocket communications.

3.1 Core Database Schemas

Table: users (Primary DB / SQLite Fallback)

Field Name	Type	Description
id	Integer (PK)	Unique database sequence id.
username	String	Candidate username (unique key).
password	String	Encrypted password hash.
status	String	Candidate application state (Approved, Pending, Rejected).
access	String	Access flag (grant, revoke) toggled by administrator.
resume_path	String	FSR path or Cloudinary URL to uploaded resume PDF.
skills	String	Comma-separated list of parsed candidate skills.
email / phone	String	Contact details extracted from resume parsing.
integrity_notes	String	Timestamped list of focus violations during simulation.
full_video_url	String	Cloud/Local path to complete interview video recording.

Table: interview_sessions (Primary DB / SQLite Fallback)

Field Name	Type	Description
id	Integer (PK)	Unique interview session record sequence.
username	String (FK)	References users.username to map candidates.
date	String	Timestamp when question answer took place.
question	String	Prompt presented to candidate.
answer	String	Speech-to-text transcript generated.
emotion	String	Dominant candidate emotion during this question.
score	Float	Numeric evaluation rating between 0 and 100.
video_url	String	Path/URL of this specific answer's video recording.
evaluation_feedback	String	Detailed text feedback from Gemini / offline grading.

3.2 Core API Endpoint Catalog

- **POST /api/register:** Registers new candidate accounts. Validates username uniqueness and stores metadata.
- **POST /api/login:** Checks credentials. Issues a JWT security access token configured for 24-hour expiration.
- **POST /api/upload_resume:** Accepts resume multipart uploads. Invokes the PDF parser, extracts candidate metrics, classifies domain, and returns customized questions.
- **POST /api/analyze_answer:** Receives answers. Translates webcam recordings or speech audio bytes, evaluates content via Gemini/TF-IDF, and commits a session row to the database.
- **GET /api/get_result:** Enables candidates to fetch their status (Approved/Rejected), average scoring, cheating logs, and custom AI feedback comments.
- **GET /api/get_all_records:** Admin-only call. Gathers all user and session data. Dynamically compiles transcripts and fetches localized recordings. Synchronizes and merges data on the fly.
- **POST /api/admin_action:** Allows managers to approve/reject applicants, grant or revoke simulation privileges, and purge candidate dossiers.
- **POST /api/log_violation:** Client-side trigger. Append cheating events (e.g. eye-gaze drifting or tab blurring) to user integrity logs in real-time.
- **POST /api/webrtc/offer:** Establish WebRTC connection parameters to record remote webcam streams on host containers using `aiortc` structures.

4. Deployment, Operations & Testing

The system supports multiple operational modes to fit both light staging tests and heavy production loads:

- **Local Development (SQLite & Threading):** Quickstart configuration. Uses local SQLite and threadpools to handle uploads. Avoids external dependencies like Docker or Redis, making testing quick and cost-free.
- **Production Scale (Docker, Postgres, Celery & Redis):** Utilizes a four-container stack managed via `docker-compose.yml`: (1) a `web` FastAPI container, (2) a `db` PostgreSQL database, (3) a `redis` broker for queuing tasks, and (4) a `worker` Celery queue. Whisper transcriptions are completed asynchronously on the worker container, preventing main thread locks during heavy candidate uploads.
- **Database Migration Logic:** A utility script `migrate\_to\_postgres.py` connects to SQLite, extracts local candidates and session tables, cleans sequence indices, and loads them into PostgreSQL, ensuring simple staging-to-production migration.
- **Cloud Provider Integration:** Cloudinary integration automatically uploads resumes and session clips. Includes local file system backups if credentials are not configured.

4.1 Verification & Validation Plan

System reliability was verified across multiple vectors: (1) Unit tests for checking resume parsing correctness, (2) Edge-case validations confirming automatic fallback from Gemini to TF-IDF cosine scoring when API keys are absent, (3) Database locking tests confirming dual DB reading capabilities, and (4) Manual browser testing of client-side face calibration and WebRTC transcription reliability in local Chrome and Firefox setups.

End of Technical Summary Report